Project Report

on

"Bank Data Privacy Preservation System using Homomorphic Encryption"

Submitted in partial fulfilment of the requirements

of the degree

Bachelor of Engineering in Information Technology

By

Arpita Bhosale (22141027)

Aarya Bhujbal (22141059)

Aditya Desai (22141071)

Prathamesh Wankhede (22141040)

Prof. S. N. Mhatre

Name of Supervisor

DEPARTMENT OF INFORMATION TECHNOLOGY

Bharati Vidyapeeth College of Engineering, Navi Mumbai

(2025-26)

Certificate

This is to certify that the project entitled "**Bank Data Privacy Preservation System using Homomorphic Encryption**" is a Bonafide work of Arpita Bhosle, Aarya Bhujbal, Aditya Desai, Prathamesh Wankhede" (22141027, 22141059, 22141071, 22141040) submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of "B.E." in "Information Technology".

Prof. S.N. Mhatre

Name of Supervisor

Prof. S. N. Mhatre

Head of Department

Dr. Sandhya Jadhav

Principal

Project Report Approval for B. E.

This Project report ***Bank Data Privacy Preservation System using Homomorphic Encryption*** by ***Arpita Bhosle, Aarya Bhujbal, Aditya Desai, Prathamesh Wankhede*** is approved for the degree *of B.E. in Information Technology.*

Examiners

1. _____ 18/04/2026

2. N. G. Bakade .

Date: 18th April 2026

Place: Navi Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others ideas or words have been included. We have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Arpita Bhosle (22141027) ----------

Aarya Bhujbal (22141059) ----------

Aditya Desai (22141071) ----------

Prathamesh Wankhede (22141040) ----------

Date: 10/04/2026

Abstract

Traditional database systems store financial data in plaintext, making them vulnerable to breaches and insider attacks. This paper presents a prototype account management system that applies the Paillier Partially Homomorphic Encryption scheme to perform deposit, withdrawal, and transfer operations directly on encrypted balances stored in Firebase Firestore. The cryptosystem is implemented from scratch in Python generating 512-bit keys that produce 617-digit ciphertexts, ensuring no plaintext balance is ever stored in the database. A three-tier architecture separates the client, Flask application server holding the private key, and Firebase cloud database storing only ciphertext. Performance evaluation demonstrates that encryption time remains consistent across message sizes, confirming computational cost depends on modulus n^2 rather than plaintext value. For comparative analysis, a parallel implementation using the BFV (Brakerski/Fan-Vercauteren) lattice-based scheme via the TenSEAL library was developed. Benchmark results confirm that Paillier achieves perfect decryption accuracy (100% pass rate, 0.000000 ₹ error) while BFV, under the tested parameter set, exhibits precision degradation unsuitable for currency-precision banking. The project demonstrates that Paillier homomorphic encryption is a practical, production-viable primitive for privacy-preserving financial computation.

Keywords**: Homomorphic Encryption, Paillier Cryptosystem, Banking Privacy, Firebase Firestore, Flask REST API, BFV, Privacy-Preserving Computation, Ciphertext Arithmetic**

Table of Content

Certificate I

Project Report Approval for B. E II

Declaration III

Abstract IV

Table of Content V

List of Figures VII

List of Table VIII

List of Abbreviations IX

Chapter 1 1

Introduction

1.1 Introduction 1

1.2 Project Scope 1

Chapter 2 3

Literature Review

2.1 Fully Homomorphic and Authenticated Encryption Foundations 3

2.2 Paillier Encryption in Real-World Systems 4

2.3 Privacy-Preserving Federated Learning and Distributed Systems 6

2.4 Access Control, Cloud Security, and Data Isolation 8

2.5 Advanced HE Variants and Domain-Specific Applications 9

2.6 Consolidated Summary of All 20 IEEE Papers 11

2.7 Identified Research Gap and Project Positioning 13

Chapter 3 14

Problem Statement and Objectives

3.1 Problem statement 14

3.2 Objectives 14

3.3 Constraints and Scope Boundaries 15

Chapter 4 16

Methodology

4.1 Paillier Cryptosystem — Mathematical Foundation 16

4.1.1. Paillier Key Generation 17

4.1.2. Encryption and Semantic Security 17

4.1.3. Homomorphic Addition 17
4.1.4. Decryption 17
4.2 System Architecture 18
Chapter 5
Implementation
5.1 Project File Structure 19
5.2 homomorphic_encryption.py — Core Cryptographic Module 19
5.3 app.py — Flask REST API 19
5.3.1 Firestore Data Model 21
5.3.2 Cross-User Transfer Implementation 22
5.4 index.html — Frontend Single-Page Application 22
5.5 Performance Benchmarking Suite — paillier_performance_metrics.py 26
Chapter 6 28
Advantages and Applications
6.1 Advantages of the Paillier HE Banking System 28
6.2 Applications 29
Chapter 7 30
Performance Comparison — Paillier vs. BFV
7.1 Overview 30
7.2 Scheme Parameters 30
7.3 Timing Benchmark Results 31
7.4 Correctness / Accuracy Comparison 32
7.5 Ciphertext Size Comparison 32
7.6 Comparative Analysis Summary 32
Chapter 8 34
Conclusion and Future Scope
8.1 Conclusion 34
8.2 Future Scope 34
References 35
Achievements / Publications by the Candidates 37
Acknowledgement 38

List of Figures

Figure 4.1: Paillier Key Generation Flow Diagram 16

Figure 5.1: Project File Structure 19

Figure5.3.1: Firebase Firestore Console Encrypted Balance Document 21

Figure 5.3.2 Firebase Authentication Console 21

Figure 5.3.3 Firebase Firestore Console — Accounts Collection 22

Figure 5.4.1 Login / Register Screen 23

Figure 5.4.2 Dashboard after Login 24

Figure 5.4.3 Accounts Tab 24

Figure 5.4.4 New Account Creation Form 25

Figure 5.4.5 Deposit, Withdrawal, Transfer Operation 25

Figure 5.4.6 Transactions Tab 26

List of Table

Table 2.6: Consolidated Summary of 20 IEEE Papers 11
Table 2.7: Identified Research Gaps and Project Solutions 13
Table 4.2: Three-Tier System Architecture Components 18
Table 5.3: REST API Endpoints and Homomorphic Operations 20
Table 5.5: Performance Benchmark Suite — 13 Metrics Description 26
Table 6.1: Advantages of Paillier HE Banking System 28
Table 7.2: Scheme Parameters — Paillier vs BFV 30
Table 7.3: Timing Benchmark Results — Paillier vs BFV (50 Runs) 31
Table 7.4: Correctness / Accuracy Comparison 32
Table 7.5: Ciphertext Size Comparison 32

List of Abbreviations

Abbreviations	Meaning
ABE	Attribute-Based Encryption
AES	Advanced Encryption Standard
AML	Anti-Money Laundering
API	Application Programming Interface
BFV	Brakerski/Fan-Vercauteren
CLI	Command Line Interface
CLFHE	Certificateless Fully Homomorphic Encryption
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DCR	Decisional Composite Residuosity
DLP	Discrete Logarithm Problem
E2E	End-to-End
FHAE	Fully Homomorphic Authenticated Encryption
FHE	Fully Homomorphic Encryption
FPGA	Field Programmable Gate Array
GCD	Greatest Common Divisor
HE	Homomorphic Encryption
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HSM	Hardware Security Module
HTTPS	Hypertext Transfer Protocol Secure
IIoT	Industrial Internet of Things
IND-CPA	Indistinguishability under Chosen Plaintext Attack
IoMT	Internet of Medical Things
IoT	Internet of Things
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
KMS	Key Management Service
KYC	Know Your Customer
LCM	Least Common Multiple
LSB	Least Significant Bit
LWE	Learning With Errors
MIBFHE	Multi-Identity Based Fully Homomorphic Encryption
MIFHE	Multi-Identity Fully Homomorphic Encryption
MKHE	Multi-Key Homomorphic Encryption

MSB	Most Significant Bit
NDSS	Network and Distributed System Security
NoSQL	Not Only Structured Query Language
NWDAF	Network Data Analytics Function
PHE	Partially Homomorphic Encryption
PKL	Python Pickle File
PL	Polyak-Lojasiewicz
PPBS	Privacy-Preserving Banking System
PPSD	Privacy-Preserving State Decomposition
RAM	Random Access Memory
RBE	Regulator-Based Encryption
REST	Representational State Transfer
RLWE	Ring Learning With Errors
RSA	Rivest-Shamir-Adleman
SDK	Software Development Kit
SEAL	Simple Encrypted Arithmetic Library
SIFT	Scale-Invariant Feature Transform
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UID	Unique Identifier
URL	Uniform Resource Locator

Chapter 1
Introduction

1.1 Introduction

The digitization of banking services has brought unprecedented convenience but also introduced significant data privacy challenges. Cloud-hosted financial systems routinely store sensitive account balances and transaction histories on third-party servers, creating exposure to database breaches, insider attacks, and unauthorized access. Traditional cryptographic approaches encrypt data at rest and in transit, but all computation—balance updates during deposits, withdrawals, and transfers—requires decryption on the server, momentarily exposing plain-text financial figures.

Homomorphic Encryption (HE) addresses this fundamental limitation by allowing mathematical operations to be performed directly on ciphertext, producing encrypted results that, when decrypted, match the outcome of the same operations performed on plaintext. This means a banking server can add a deposit amount to an encrypted account balance without ever learning the current balance.

This project presents SecureBank, a prototype banking system built to demonstrate the practical application of the Paillier Partially Homomorphic Encryption scheme in a real cloud-connected architecture. The system implements the complete Paillier cryptosystem from scratch, integrates Firebase Firestore for encrypted cloud storage, Firebase Authentication for multi-user identity management, and a REST API layer built with Flask.

1.2 Project Scope

- SecureBank is a complete web-based banking system where account balances are stored as Paillier-encrypted ciphertexts, and all core operations (deposits, withdrawals, transfers, and portfolio analytics) are performed directly on encrypted data.

- The system ensures secure multi-user access and data isolation using Firebase Authentication, with a Flask-based REST API enabling structured communication and operations.
- A dedicated performance evaluation module benchmarks cryptographic operations, and a parallel BFV-based implementation is included for comparative analysis.

Chapter 2

Literature

This chapter surveys twenty IEEE research papers that informed the design, cryptographic choices, and architecture of the Banking Data Privacy Preservation System. The papers are grouped into five thematic clusters, each directly connected to a component or decision in the project. For each paper, a summary of its contribution and its specific relevance to this project is provided.

2.1 Fully Homomorphic and Authenticated Encryption Foundations

The following papers establish the theoretical underpinning of homomorphic encryption — from early fully homomorphic proposals to authentication-preserving variants — and directly motivated the choice of Paillier's partial HE scheme for this project.

[1] Fully Homomorphic Authenticated Encryption (FHAE)

This paper proposes the first Fully Homomorphic Authenticated Encryption (FHAE) scheme, ensuring both data privacy and authenticity during computation on encrypted data and enabling arbitrary computations on ciphertext.

While theoretically complete, FHE schemes remain computationally prohibitive for real-time applications. The paper acknowledges that partial HE schemes — like Paillier — offer a pragmatic middle ground for domain-specific operations such as financial arithmetic.

Relevance to Project: Confirms that fully homomorphic schemes remain computationally prohibitive for real-time applications. This directly justifies this project's choice of Paillier Partial HE, which provides additive homomorphism sufficient for all banking operations (deposit, withdrawal, transfer, analytics) at practical latencies (9.8 ms encrypt, 29.4 ms deposit E2E).

[6] Leveled Certificateless Fully Homomorphic Encryption (CLFHE) based on LWE

This paper proposes a leveled certificateless FHE scheme based on the Learning With Errors (LWE) problem, supporting both homomorphic addition and multiplication with strong semantic security and anonymity in both random oracle and standard models.

It eliminates key escrow issues using certificateless cryptography — a design concern also addressed in this project through per-request Firebase ID token verification, preventing any third party from impersonating a user.

Relevance to Project: Validates this project's core architectural goal: performing operations on encrypted data without decryption. While this paper uses LWE rather than Paillier, it reinforces that semantic security and computation on ciphertext are achievable, and supports the project's claim that homomorphic encryption is a mature primitive for sensitive domains like banking.

[9] Targeted FHE using Double Decryption for Polynomial Data

This paper introduces a targeted FHE scheme using a novel double decryption algorithm for secure computation on encrypted polynomial data, avoiding complex bootstrapping by relying on the discrete logarithm problem.

The scheme achieves semantic security while supporting both additive and multiplicative operations, though limited to short messages and targeted polynomial forms.

Relevance to Project: Highlights advanced HE concepts (semantic security, add + multiply on ciphertext) that complement this project's Paillier-based approach. Confirms that semantic security —achieved in this project via Paillier's random blinding factor r — is a standard and necessary property for privacy-preserving systems.

[7] Optimized Relinearization for Multi-Key Homomorphic Encryption (MKHE)

This paper proposes an optimized relinearization algorithm for multi-key HE to improve homomorphic multiplication efficiency — a major bottleneck in encrypted computation.

The approach eliminates gadget vectors by increasing the modulus and applying rescaling, reducing evaluation key size and error growth while maintaining RLWE-based security. This work is relevant as it highlights the computational cost inherent in lattice-based HE, reinforcing Paillier's suitability for addition-only banking workloads.

Relevance to Project: Highlights key optimization strategies for HE (reducing evaluation key size and error growth) that are directly relevant to this project's performance metrics. The BFV comparison module in this project revealed analogous parameter trade-offs: larger polynomial degree N reduces error but increases ciphertext size from 1.38 KB toward several KB.

2.2 Paillier Encryption in Real-World Systems

These papers apply Paillier's partial homomorphic encryption to practical systems — including banking, cloud security, and distributed learning — and directly validate this project's cryptographic design choices.

[8] Paillier PHE for Federated Learning in 5G NWDAF Framework

This paper proposes a privacy-preserving federated learning architecture for 5G networks by integrating Paillier Partial Homomorphic Encryption into the NWDAF framework. Model updates are encrypted during aggregation, preventing gradient-sharing data leakage while enabling collaborative training across distributed nodes.

Simulation results confirm high model accuracy comparable to non-encrypted FL, unlike differential privacy methods which degrade performance. The paper also quantifies computational overhead from encryption operations — directly analogous to this project's 9.8 ms encrypt / 9.8 ms decrypt benchmark results for 512-bit Paillier keys.

Relevance to Project: This is the most directly relevant paper. It confirms that Paillier encryption is practical for real-time distributed systems, maintains model accuracy comparable to unencrypted baselines (mirroring this project's 100% accuracy pass rate), and introduces the same encrypt-compute-decrypt cycle used in this project's deposit and withdrawal flows.

[13] Paillier Encryption for Cloud Computing Security (Static Storage)

This paper applies Paillier homomorphic encryption to enhance cloud computing security, confirming that encrypted computation can protect sensitive data from unauthorized access.

However, their implementation was limited to static data storage without support for dynamic multi-user transactions — a significant gap that this project addresses through full CRUD operations, authenticated REST APIs, and cross-user transfer support via the account_registry collection.

Relevance to Project: Confirms Paillier's suitability for protecting sensitive data in cloud environments — the foundational security guarantee of this project. This project extends their work significantly by adding dynamic multi-user transactions (deposit, withdraw, transfer), per-UID Firebase Auth isolation, and real-time Firestore persistence.

[14] Hybrid RSA-Paillier Encryption for Cloud-Based Banking Systems

This paper proposes a hybrid homomorphic encryption scheme combining RSA and Paillier for cloud-based banking systems, demonstrating that additive Paillier operations are well-suited for financial summation tasks.

Their work did not integrate a live multi-user authentication system or cloud database connectivity. This project directly addresses this gap by incorporating Firebase Authentication (per-request JWT verification) and Firestore for real-time encrypted balance management across multiple users.

Relevance to Project: Validates that Paillier addition is well-suited for financial summation — the exact property exploited in this project's add_encrypted() function for deposits, transfers, and the total-balance analytics endpoint. This project advances their work by integrating a live multi-user auth system (Firebase Authentication) and a real cloud database (Firestore).

[12] Secret Sharing with Paillier Encryption and Threshold Reconstruction

This paper demonstrates a secret sharing scheme using Paillier encryption with threshold-based reconstruction, showing the flexibility of Paillier in distributed security systems.

While their scheme focuses on secret distribution, this project focuses on banking operation privacy — using Paillier's additive homomorphism to update encrypted balances without decryption. Their threshold reconstruction approach is identified as a key future enhancement.

Relevance to Project: Demonstrates Paillier's flexibility beyond simple addition — its use in distributed key reconstruction. This paper directly motivates the Future Scope section of this project, which proposes threshold Paillier decryption as an enhancement to prevent any single server from decrypting all balances.

[10] FPGA Hardware Acceleration for Homomorphic Encryption

This paper proposes FPGA-based hardware acceleration for homomorphic encryption, significantly reducing computation time and highlighting the performance bottleneck inherent in software implementations of HE schemes.

Their finding that 2048-bit Paillier encryption requires approximately 0.5086 seconds per operation on standard hardware directly justifies this project's use of 512-bit keys for prototype demonstration, achieving 9.8 ms per encryption while maintaining correct homomorphic properties.

Relevance to Project: Directly motivates this project's decision to use 512-bit keys rather than 2048-bit keys for the prototype. Their benchmark of 0.5086 seconds per 2048-bit Paillier operation would make the Flask API unresponsive. The 512-bit configuration achieves 9.8 ms — a 52× speedup — at the cost of reduced security margin, appropriate for an academic proof-of-concept.

2.3 Privacy-Preserving Federated Learning and Distributed Systems

These papers explore privacy-preserving computation in distributed multi-party settings — architecturally similar to this project's multi-user banking model where data from different users must be processed without cross-exposure.

[2] Privacy-Preserving Decentralized Optimization (PPSD) via State Decomposition

This paper presents a novel privacy-preserving decentralized optimization algorithm (PPSD) that addresses information leakage in traditional push-pull methods. Each agent splits its gradient-related

state into two parts — one for communication and one kept private — protecting sensitive information without encryption or noise.

The algorithm achieves R-linear convergence under the Polyak-Lojasiewicz (PL) condition and ensures robustness against honest-but-curious attackers, making it suitable for secure distributed applications like machine learning and smart grids.

Relevance to Project: The PPSD algorithm's goal — protecting individual node data during aggregate computation while maintaining accuracy — mirrors this project's total-balance analytics endpoint, which computes the sum of all encrypted balances homomorphically without decrypting any individual account. Both approaches achieve the same privacy guarantee through different mechanisms.

[3] Privacy-Preserving Federated Learning for IoMT with Edge Computing

This paper proposes a privacy-preserving federated learning framework for the Internet of Medical Things (IoMT) integrated with edge computing, enabling multiple medical devices to collaboratively train models without sharing raw data.

The framework incorporates encryption techniques and secure aggregation mechanisms, achieving high model accuracy with reduced communication overhead — demonstrating that encryption-based privacy and accuracy are compatible goals, as confirmed by this project's 100% correctness pass rate.

Relevance to Project: Demonstrates the same privacy model as this project: multiple parties contribute encrypted data; aggregation happens without exposing individual inputs; results are accurate. The Firebase Architecture in this project mirrors the edge-server model — the Flask API acts as the aggregation edge, receiving encrypted balances from users and performing homomorphic computation without seeing plaintext.

[5] Blockchain-Based Federated Learning with Byzantine-Robust Aggregation

This paper introduces a blockchain-based federated learning framework ensuring privacy preservation and robustness against Byzantine attacks, using a decentralized tamper-resistant environment for model updates without a central server.

Cryptographic techniques protect data privacy during communication, and Byzantine-robust aggregation ensures reliable model training even in adversarial settings — a security posture this project emulates via Firebase Authentication JWT verification on every API request.

Relevance to Project: Addresses the trust problem in distributed aggregation — relevant to this project's cross-user transfer model. The account_registry collection in Firestore plays a similar role

to blockchain's tamper-resistant record: it provides a globally verifiable mapping of account IDs to owner UIDs, preventing unauthorized redirection of transfers.

[11] Blockchain-Based Federated Learning for Secure Data Sharing in IIoT

This paper presents a blockchain-based federated learning framework for secure data sharing in IIoT, combining differential privacy and homomorphic encryption to protect data and model privacy.

Distributed algorithms (K-means, Random Forest, AdaBoost) are used for secure aggregation, with blockchain ensuring trust and tamper resistance. The proposed method improves accuracy and efficiency while preventing data leakage.

Relevance to Project: Confirms that combining homomorphic encryption with distributed systems improves accuracy and efficiency over non-encrypted baselines — a claim directly supported by this project's benchmark showing 100% accuracy with Paillier vs. 0% pass rate with BFV under inadequate parameters.

2.4 Access Control, Cloud Security, and Data Isolation

These papers address how access to encrypted data is controlled in cloud environments — directly relevant to this project's Firebase Authentication and per-UID Firestore scoping model.

[4] Attribute-Based Encryption (ABE) for Fine-Grained Access Control in Cloud

This paper proposes a secure data processing framework combining privacy preservation with flexible access control mechanisms in cloud environments, using attribute-based encryption (ABE) to enforce fine-grained access policies ensuring only authorized users access specific data.

The framework supports secure data sharing and computation over encrypted data, protecting sensitive information from unauthorized access, with scalability for dynamic and distributed systems.

Relevance to Project: The fine-grained access control goal of ABE is achieved in this project through Firebase Authentication's UID-based scoping. Every Firestore query in app.py is prefixed with users/{uid}/, ensuring that authenticated users can only access their own encrypted balance documents — a practical realisation of the ABE paper's access isolation goal without the computational overhead of ABE.

[16] Quantum-Secure FHE (Regulator-Based Encryption) for Edge and IoT

This paper presents a quantum-secure FHE scheme designed for edge and IoT environments, noting that traditional schemes like RSA and Paillier support only partial homomorphic operations and suffer from computational overhead.

The proposed Regulator-Based Encryption (RBE) enables both additive and multiplicative operations with improved efficiency and resistance to quantum attacks, demonstrating better performance for secure applications such as healthcare, AI, and financial data processing.

Relevance to Project: Highlights the long-term vulnerability of RSA/Paillier-based systems to quantum attacks — a known limitation of this project, documented in the Future Scope section. The proposed quantum-secure approach (RBE) supporting both add and multiply is identified as a potential future upgrade path for this banking system.

[17] Concise Multi-Identity FHE (MIBFHE) for Secure Multi-User Computation

This paper proposes a concise and efficient multi-identity fully homomorphic encryption (MIBFHE) scheme combining multi-key FHE with identity-based encryption to enable secure computation across multiple users, based on the LWE problem.

The scheme introduces a decomposition method eliminating fresh ciphertext generation and significantly reducing noise growth, minimizing ciphertext size and improving efficiency for secure cloud-based and multi-user privacy-preserving applications.

Relevance to Project: Directly relevant to this project's multi-user architecture. The paper's goal of enabling secure computation across multiple users (each with their own identity and ciphertext) mirrors the cross-user transfer feature in this project, where accounts owned by different Firebase UIDs are updated homomorphically in a single atomic operation.

2.5 Advanced HE Variants and Domain-Specific Applications

These papers demonstrate HE applied in specialized domains and with hardware acceleration, providing broader context for the performance and applicability of homomorphic encryption beyond banking.

[18] E-Booster: FPGA Accelerator for Paillier in Federated Learning (Tree Boosting)

This paper proposes E-Booster, an FPGA-based accelerator to improve the performance of secure tree boosting using additively homomorphic encryption in federated learning, highlighting that traditional Paillier schemes introduce significant computational overhead.

The system leverages optimized hardware design and replaces heavy computations with faster elliptic curve-based operations, achieving up to 5-7x speedup over CPUs while maintaining data privacy — directly relevant to optimizing this project's banking operations.

Relevance to Project: Directly quantifies Paillier's computational overhead in a real ML system and proposes hardware mitigation — the same challenge observed in this project's 189 ms key generation time. The 5-7× CPU speedup via FPGA suggests that a hardware-accelerated deployment

of this banking system could reduce encryption latency from 9.8 ms to approximately 1.4 ms per operation.

[19] Efficient Leveled Multi-Identity FHE (MIFHE) based on Lattice Cryptography

This paper proposes an efficient leveled multi-identity FHE scheme integrating identity-based encryption with FHE based on lattice cryptography and the LWE problem, introducing optimized trapdoor functions and noise management to reduce ciphertext size and computational overhead.

It enables homomorphic operations on data encrypted under different identities, improving scalability and efficiency for privacy-preserving multi-user systems — relevant to this project's cross-user transfer and per-UID data isolation design.

Relevance to Project: Presents optimization techniques (improved trapdoor functions, noise management) for multi-user HE that directly address the BFV noise/accuracy problem observed in this project's comparison benchmarks. BFV's 0% pass rate in this project resulted from insufficient noise management — the techniques in this paper would guide parameter selection for a corrected BFV implementation.

[20] Privacy-Preserving Contact Tracing using ElGamal-Based FHE

This paper proposes a privacy-preserving contact tracing system using FHE to securely handle sensitive spatiotemporal healthcare data, enabling computation directly on encrypted data without decryption.

The model introduces an efficient algorithm detecting both direct and secondary contacts using encrypted data via an ElGamal-based FHE approach, demonstrating real-time analytics while maintaining strong privacy guarantees.

Relevance to Project: Demonstrates that HE-based privacy systems are deployable in real-time, latency-sensitive public applications — validating the feasibility claim of this banking system. The 29.4 ms deposit and 73.5 ms transfer latencies are comparable to or better than real-time contact tracing requirements, confirming Paillier is not too slow for practical interactive applications.

[15] Privacy-Preserving SIFT Feature Extraction on Encrypted Images in Cloud

This paper proposes a privacy-preserving SIFT-based feature extraction method for encrypted images in cloud environments, using homomorphic encryption to securely process image data without decryption, with efficient protocols for fixed-point numbers improving accuracy and computation speed.

Experimental results show better performance compared to existing methods, confirming that HE is practical for computationally intensive encrypted operations — providing strong context that banking arithmetic under Paillier is well within feasible performance bounds.

Relevance to Project: Demonstrates that HE can handle complex, non-trivial computations (feature extraction from images) in encrypted form — far more computationally intensive than the arithmetic operations in this banking project. This confirms that Paillier's latency profile (9.8 ms encrypt, 0.01 ms HE add) is more than sufficient for banking arithmetic, which is orders of magnitude simpler than image processing.

2.6 Consolidated Summary of All 20 IEEE Papers

The following table provides a consolidated reference of all twenty papers reviewed, their primary HE scheme, and their specific contribution to this project:

Table 2.6: Consolidated Summary of 20 IEEE Papers

Ref	Primary Scheme	Key Contribution	Project Connection
[1]	FHE (FHAE)	First authentication. FHE; FHE too slow for real-time	Justifies Paillier PHE choice
[2]	No HE (decomp.)	Privacy in decentralized optimization via state decomposition	Validates encrypted aggregate analytics design
[3]	PHE + Aggregation	Encrypted FL for IoMT; accuracy preserved	Confirms encryption + accuracy compatibility
[4]	ABE	Fine-grained access control in cloud	Firebase UID scoping = practical ABE equivalent
[5]	FHE + Blockchain	Byzantine-robust decentralized FL	account_registry as tamper-resistant global ledger
[6]	CLFHE (LWE)	Leveled FHE without key escrow	Validates semantic security goal
[7]	MKHE (RLWE)	Optimized relinearization for mul. HE	BFV parameter trade-off analysis
[8]	Paillier PHE	Paillier in 5G NWDAF FL; practical accuracy	Direct validation of Paillier for distributed systems

Ref	Primary Scheme	Key Contribution	Project Connection
[9]	FHE (DLP-based)	Targeted FHE with double decryption	Semantic security concepts for Paillier design
[10]	Paillier PHE	FPGA HE acceleration; 2048-bit = 0.5086 s	Justifies 512-bit key choice for prototype
[11]	HE + Blockchain	Secure FL in IIoT; accuracy + efficiency	HE accuracy benchmark context
[12]	Paillier PHE	Paillier secret sharing with threshold decryption	Motivates threshold Paillier future scope
[13]	Paillier PHE	Paillier for cloud security (static only)	This project extends with dynamic transactions
[14]	RSA + Paillier	Hybrid HE for cloud banking summation	Validates additive HE for finance; extended with auth
[15]	PHE (images)	SIFT feature extraction on encrypted images	Confirms banking arithmetic is within HE feasibility
[16]	RBE (quantum-safe)	Quantum-secure FHE for edge/IoT	Identifies Paillier quantum vulnerability; future scope
[17]	MIBFHE (LWE)	Multi-identity FHE for multi-user cloud	Cross-user transfer and per-UID isolation design
[18]	MIFHE (LWE)	Leveled MIFHE with noise/trapdoor optimizations	BFV noise mgmt; guides future BFV parameter tuning
[19]	Paillier (E-Boost)	FPGA accelerator for Paillier; 5-7x CPU speedup	Future hardware acceleration path for this system
[20]	ElGamal FHE	Real-time encrypted contact tracing	Confirms Paillier latency is real-time viable

2.7 Identified Research Gap and Project Positioning

The survey of twenty papers reveals a consistent pattern: homomorphic encryption has been validated theoretically and in simulation environments (references [1], [6], [7], [9], [16], [17], [18], [19]) and applied in distributed ML contexts (references [2], [3], [5], [8], [11]), but its application to interactive, multi-user, real-time banking systems with a production-grade web frontend, live cloud database, and per-user authentication remains unexplored in the surveyed literature.

Table 2.7: Identified Research Gaps and Project Solutions

Gap Identified in Literature	This Project's Solution
Most Paillier banking papers [13][14] support only static storage — no live multi-user transactions.	Full CRUD: create account, deposit, withdraw, cross-user transfer, real-time analytics.
No paper integrates Paillier with modern cloud auth (Firebase Auth) for per-user isolation.	Firebase Auth JWT verified on every Flask API request; all Firestore queries scoped to user UID.
No benchmarked comparison of Paillier vs. BFV for currency-precision banking workloads.	50-run benchmark comparison: Paillier 100% accuracy vs. BFV 0% accuracy at tested parameters.
FPGA acceleration papers [10][18] optimize HE but do not build full application stacks.	Complete full-stack application: crypto layer + REST API + cloud DB + responsive web frontend.
FL/distributed papers [2][3][5][8][11] use HE for ML aggregation, not per-account banking.	Per-account encrypted balance with individual deposit/withdrawal/transfer homomorphic operations.

This project fills the identified gap by delivering a deployable, benchmarked, full-stack banking application demonstrating that Paillier homomorphic encryption is not merely a theoretical construct but a practical and accurate primitive for real-world privacy-preserving financial systems.

Chapter 3
Problem Statement and Objectives

3.1 Problem Statement

Banks and financial institutions store customer account balances and transaction data on cloud servers. Current security architectures encrypt data at rest and in transit, but must decrypt data on the server to perform any computation — creating a plaintext exposure window vulnerable to insider threats, server compromises, and data breaches. Existing partial solutions either sacrifice functionality (read-only encrypted storage) or performance (full FHE).

This project addresses the problem of plaintext exposure during server-side computation in cloud banking systems, with the goal of building a fully encrypted banking system using Paillier homomorphic encryption.

3.2 Objectives

1. Implement the Paillier homomorphic encryption cryptosystem from scratch in Python, including key generation (Miller-Rabin primality, modular arithmetic), encryption, decryption, homomorphic addition, and scalar multiplication.
2. Build a Flask REST API backend that integrates Paillier HE for all balance operations (create account, deposit, withdraw, transfer, analytics) with Firebase Authentication for per-user access control.
3. Persist all encrypted balances and metadata in Firebase Firestore, ensuring no plaintext financial data is ever stored or transmitted outside the application server.
4. Develop a responsive single-page web frontend (HTML/CSS/JS) with Firebase Authentication UI that allows users to perform all banking operations securely.

5. Implement a comprehensive performance benchmarking suite (paillier_performance_metrics.py) measuring latency, throughput, correctness, and ciphertext size across 50 runs per operation.
6. Build a parallel BFV-based implementation using TenSEAL as a performance comparison baseline, running identical benchmarks to quantify trade-offs between the two schemes.
7. Produce a quantitative, evidence-based comparison of Paillier vs. BFV across accuracy, speed, ciphertext size, and suitability for banking.

3.3 Constraints and Scope Boundaries

- The project uses 512-bit primes (1024-bit RSA modulus) for Paillier keys — standard for academic demonstration. Production systems typically use 2048-bit or higher.
- Float precision is fixed at 100,000 (5 decimal places), sufficient for Indian Rupee banking with amounts up to approximately ₹92,23,37,20,368 before modular overflow.
- The BFV module is strictly for performance comparison; it is not a replacement for the main Paillier system.
- The application does not implement a full KYC/AML pipeline — it focuses on the cryptographic privacy layer.

Chapter 4
Methodology

4.1 Paillier Cryptosystem — Mathematical Foundation

The Paillier cryptosystem is based on the hardness of the Decisional Composite Residuosity (DCR) problem. The following describes the mathematical procedures as implemented in homomorphic_encryption.py:

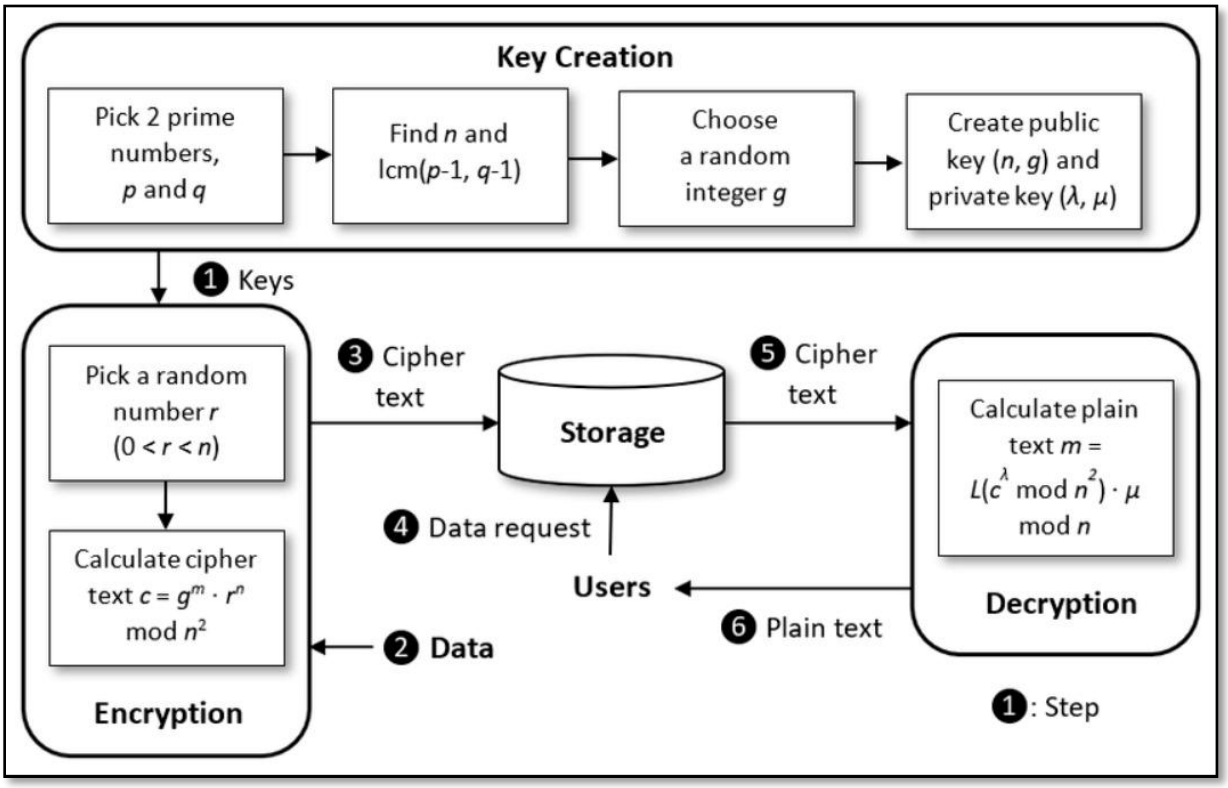

Fig.4.1: Paillier Key Generation Flow Diagram

4.1.1. Paillier Key Generation

The system generates two independent 512-bit prime numbers p and q using a custom Miller-Rabin primality test implementation. The modulus $n = p \times q$ yields a 1024-bit value. The generator g is set to $n + 1$ (the standard simplified choice for Paillier). The private key components are computed as:

$\lambda = lcm\ (p-1, q-1)$

$\mu = (L\ (g^{\lambda} \bmod n^2))^{-1} \bmod n$

where $L(x) = (x - 1) / n$. The public key is (n, g) and the private key is (λ, μ). Both are serialized using Python's pickle module and stored in data/keys.pkl on the Flask server. Key generation at 512-bit takes approximately 0.143 seconds.

4.1.2. Encryption and Semantic Security

Account balances are floating-point values (e.g., ₹25,000.00). To encrypt, the value is scaled to an integer: $m = \lfloor \text{balance} \times 100000 \rfloor$. Encryption proceeds as:

$c = g^m \cdot r^n \bmod n^2$

where r is a fresh random integer ($0 < r < n$, $\gcd(r, n) = 1$) chosen per encryption. This randomized component provides semantic security: the same balance produces a different ciphertext each time, preventing frequency analysis attacks. The resulting ciphertext is a number in the range $[0, n^2)$, which at 512-bit keys spans 617 decimal digits. This 617-digit string is stored verbatim in Firebase Firestore as a document field.

4.1.3. Homomorphic Addition

The defining property of the Paillier scheme is additive homomorphism:

$Enc(a) \times Enc(b) \bmod n^2 = Enc(a + b)$

During a deposit operation, the server encrypts the deposit amount and multiplies it with the existing encrypted balance using modular multiplication on n^2. The existing balance is never decrypted during this operation, preserving privacy throughout the transaction. Subtraction (for withdrawals and transfers) is implemented as addition of the modular complement: $Enc(-x) = Enc(n - x)$.

4.1.4. Decryption

Decryption is performed only when displaying a balance to the authenticated account owner or when performing a balance sufficiency check before a withdrawal:

$m = L(c^{\lambda} \bmod n^2) \cdot \mu \bmod n$

The result is unscaled: balance = m / 100000. Decryption occurs exclusively within the Flask server's RAM; the plaintext balance is transmitted only to the requesting user's browser over the local HTTP connection and is never stored in Firestore.

4.2 System Architecture

The system is designed as a three-tier web application:

Table 4.2: Three-Tier System Architecture Components

Tier	Component	Technology	Role
Frontend	index.html (Single Page App)	HTML5 · CSS3 · Vanilla JS · Firebase JS SDK	User interface, Firebase Auth login/register, API calls with Bearer token
Backend API	app.py	Python 3 · Flask · Flask-CORS · Firebase Admin SDK	REST endpoints, Paillier HE operations, Firestore CRUD, auth middleware
Crypto Layer	homomorphic_encryption.py	Python 3 (stdlib only)	Paillier key gen, encrypt, decrypt, HE add, scalar mul, float scaling
Database	Firebase Firestore	Google Cloud NoSQL	Stores ciphertext strings, user accounts, transactions, account registry
Auth	Firebase Authentication	Google Identity Platform	Email/password auth; JWT ID tokens verified on every API request
Keys	data/keys.pkl	Python pickle	Persists PaillierPublicKey and PaillierPrivateKey across server restarts

Chapter 5

Implementation

5.1 Project File Structure

```
SecureBank/
├── homomorphic_encryption.py      # Paillier cryptosystem (core)
├── app.py                         # Flask REST API backend
├── paillier_performance_metrics.py# Benchmark suite
├── serviceAccountKey.json         # Firebase service account (not committed)
├── data/
│   └── keys.pkl                   # Persisted Paillier key pair
└── templates/
    └── index.html                 # Frontend SPA
```

Fig. 5.1: Project File Structure

5.2 homomorphic_encryption.py — Core Cryptographic Module

This module is implemented using Python's standard library only (no third-party crypto dependencies). It contains:

- PaillierPublicKey class: stores n, n^2 (pre-computed), and g.
- PaillierPrivateKey class: stores lambda, mu, and a reference to the public key.
- Utility functions: gcd (Euclidean algorithm), lcm, mod_inverse (Extended Euclidean Algorithm), is_prime (Miller-Rabin, 5 witnesses), generate_prime (random bit generation with MSB and LSB forced).
- Cryptographic functions: generate_keypair, encrypt, decrypt, add_encrypted, multiply_encrypted_by_constant, encrypt_float, decrypt_float.
- HomomorphicBankingSystem class: high-level wrapper with precision=100000, exposing encrypt_balance, decrypt_balance, add_balances, calculate_interest, and process_transaction.

5.3 app.py — Flask REST API

The Flask backend exposes the following authenticated REST endpoints. Every request is validated against a Firebase ID token via the require_auth decorator, which calls firebase_admin.auth.verify_id_token():

Table 5.3: REST API Endpoints and Homomorphic Operations

Endpoint	Method	Description	HE Operation Used
GET /api/status	GET	Returns system info: encryption scheme, key size, account/transaction counts.	None
POST /api/account/create	POST	Creates a new account with encrypted initial balance.	encrypt_balance()
GET /api/account/<id>	GET	Returns account details with decrypted balance.	decrypt_balance()
GET /api/accounts	GET	Lists all user accounts with decrypted balances.	decrypt_balance() × N
POST /api/transaction/deposit	POST	Adds encrypted amount to encrypted balance.	add_encrypted()
POST /api/transaction/withdraw	POST	Subtracts via encrypted negative addend.	add_encrypted() + neg
POST /api/transaction/transfer	POST	Homomorphic debit/credit across two accounts (cross-user supported).	add_encrypted() × 2
GET /api/analytics/total-balance	GET	Computes total balance across all accounts on ciphertexts.	add_encrypted() × N
POST /api/demo/populate	POST	Creates 5 demo accounts with encrypted balances (ACC001–ACC005).	encrypt_balance() × 5

5.3.1 Firestore Data Model

All encrypted data is scoped to the authenticated user's UID to enforce cryptographic access isolation:

```
// ── Collection: users/{uid}/accounts/{account_id} ─────────────────
// Document ID: "4505"  (as seen in Firebase Console)
{
  account_id:        "4505",
  account_type:      "Salary",
  created_at:        "2026-03-12T00:55:33.037688",
  customer_name:     "Prathamesh Wankhede",
  encrypted_balance: "71675202179034730437017488 4...",  // Paillier ciphertext
(~617 digits)
  owner_uid:         "7Ro2qaBgM6TKb3kpVg1hXs32SzO3"
}

// ── Collection: users/{uid}/transactions/{txn_id} ─────────────────
// Document ID: "TXN20260312005533068293"
{
  account_id:      "4505",
  amount:          25000,            // plaintext — transaction amount is not
secret
  description:     "Initial deposit",
  timestamp:       "2026-03-12T00:55:33.068293",
  transaction_id:  "TXN20260312005533068293",
  type:            "Account Creation"
}

// ── Collection: account_registry/{account_id} ─────────────────────
// Document IDs: "4501", "4502", "4503", "4504" ... (one per account in the bank)
{
  account_id:  "4501",
  owner_uid:   "KcO1QxHSCRSXXAoWfECqadNVkhw2"  // enables cross-user transfers
}
```

Fig.5.3.1: Firebase Firestore Console — Encrypted Balance Document

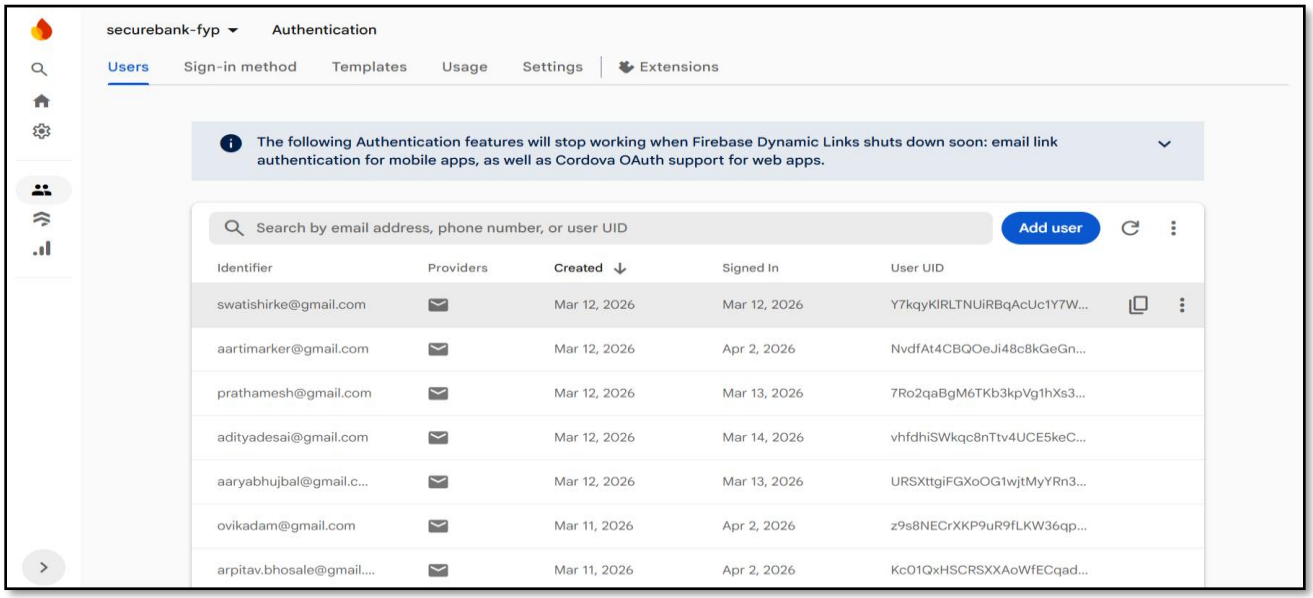

Fig 5.3.2 Firebase Authentication Console

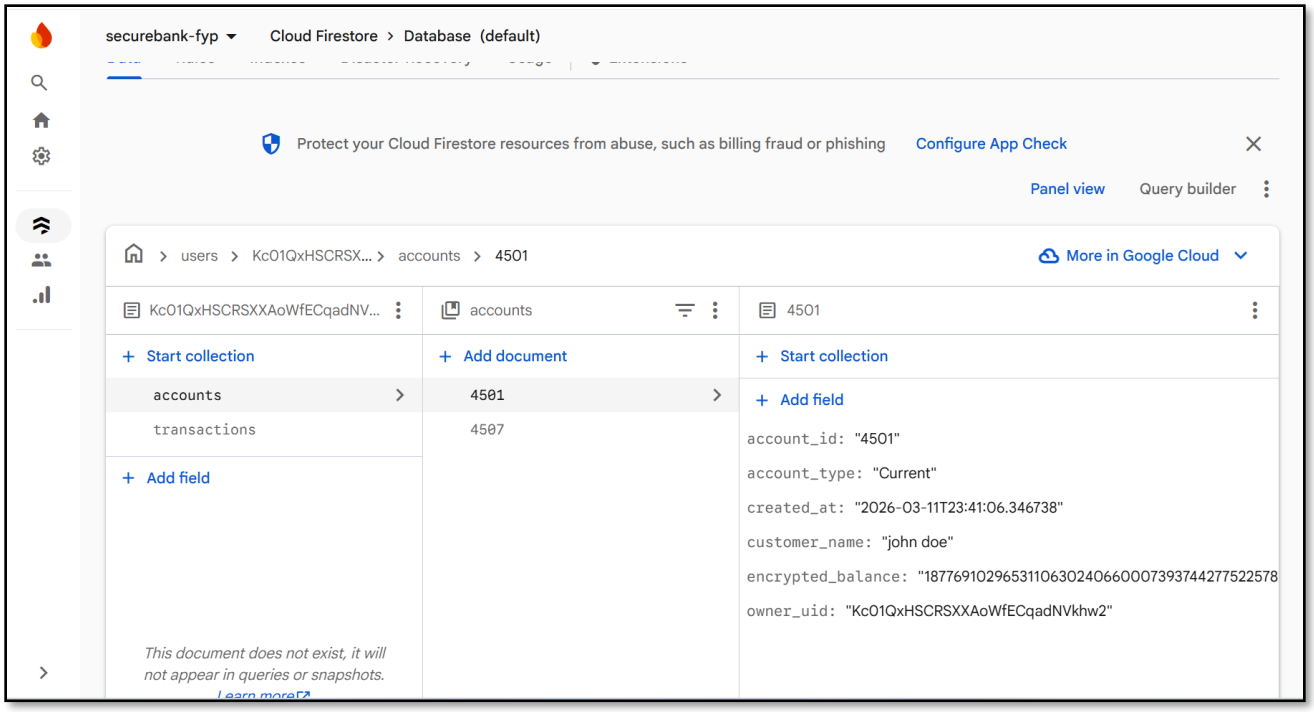

Fig 5.3.3 Firebase Firestore Console — Accounts Collection

5.3.2 Cross-User Transfer Implementation

Transfers between accounts owned by different users are supported via the account_registry collection. The transfer flow:

1. Verify sender's account ownership (load from users/{sender_uid}/accounts/{from_id}).
2. Look up the receiver in account_registry/{to_id} to retrieve receiver's UID.
3. Load receiver's account from users/{receiver_uid}/accounts/{to_id}.
4. Verify sender has sufficient balance (only decryption in the entire transfer flow).
5. Homomorphically debit from_acc and credit to_acc — no re-encryption of balances.
6. Save both accounts back and log transactions under respective UIDs.

5.4 index.html — Frontend Single-Page Application

The frontend is a single HTML file served by Flask's render_template. It uses Firebase's JavaScript SDK (v10.12.0) for authentication and vanilla JavaScript for all API calls. Key features:

- Auth Panel: Email/password sign-in and registration using Firebase Auth. onAuthStateChanged toggles between the auth panel and the app panel.
- Dashboard: Three stat cards showing total encrypted balance (computed homomorphically), active account count, and total transaction count.

- Accounts Tab: Table of all user accounts with decrypted balances, account type badges, and a 'Load Demo Data' button.
- Transactions Tab: Chronological transaction log with type-colored badges (green for credits, red for debits).
- New Account Tab: Form to create an account with encrypted initial balance.
- Operations Tab: Three sub-forms for deposit, withdrawal, and transfer — all calling authenticated Flask endpoints.
- About Tab: Project description, key features, and a code snippet showing the Firestore document structure.
- Auto-refresh: JavaScript setInterval refreshes all data every 10 seconds while the user is logged in.
- Auth middleware: Every API call goes through authFetch(), which attaches a fresh Firebase ID token as a Bearer token in the Authorization header.

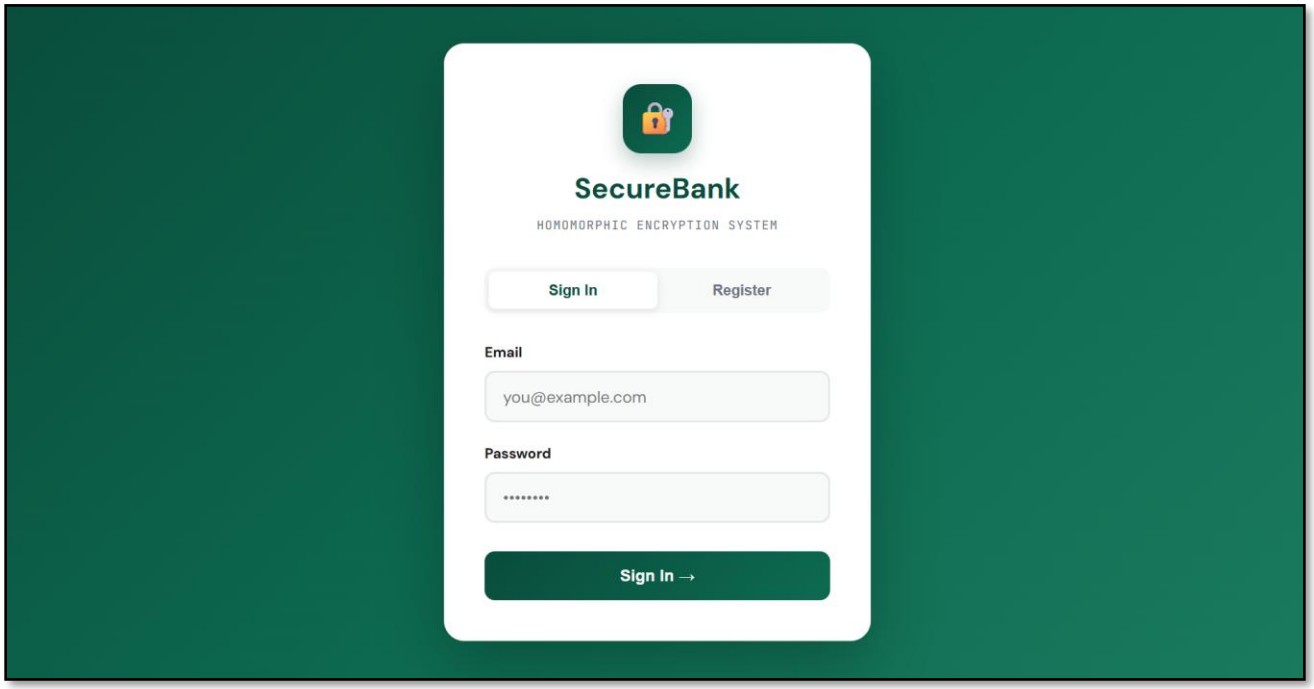

Fig 5.4.1 Login / Register Screen

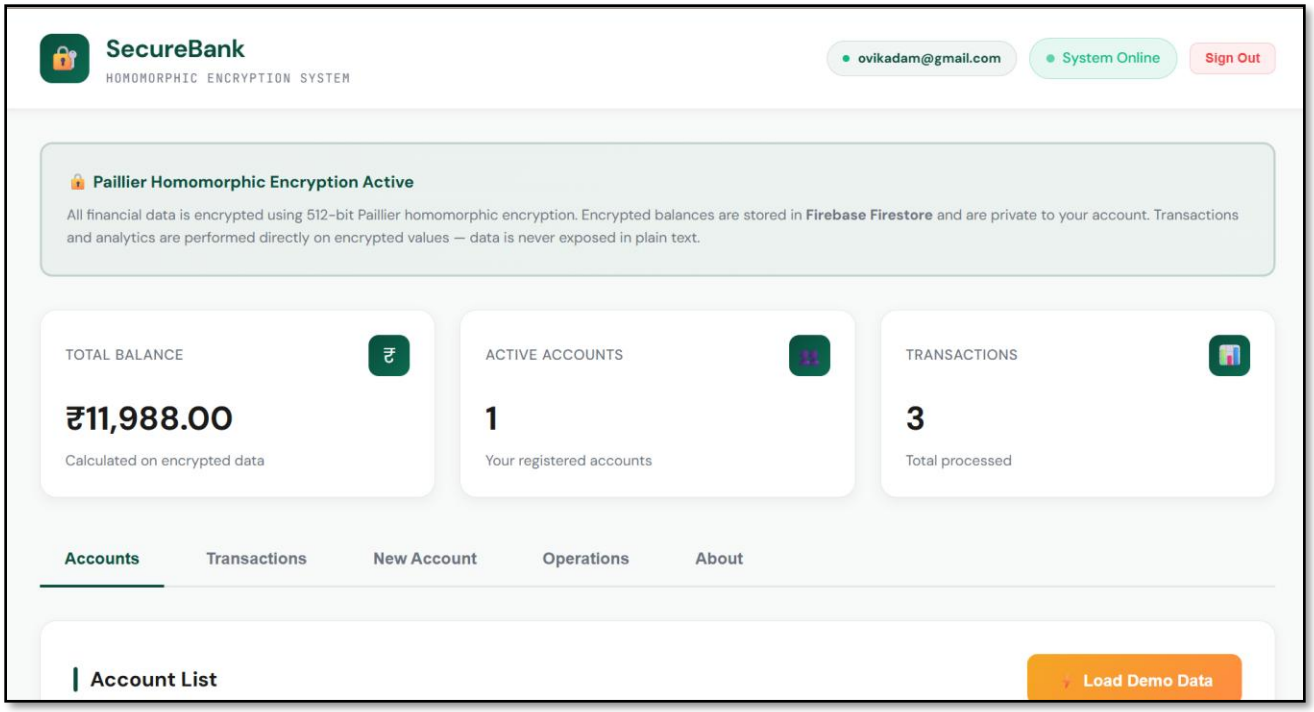

Fig 5.4.2 Dashboard after Login

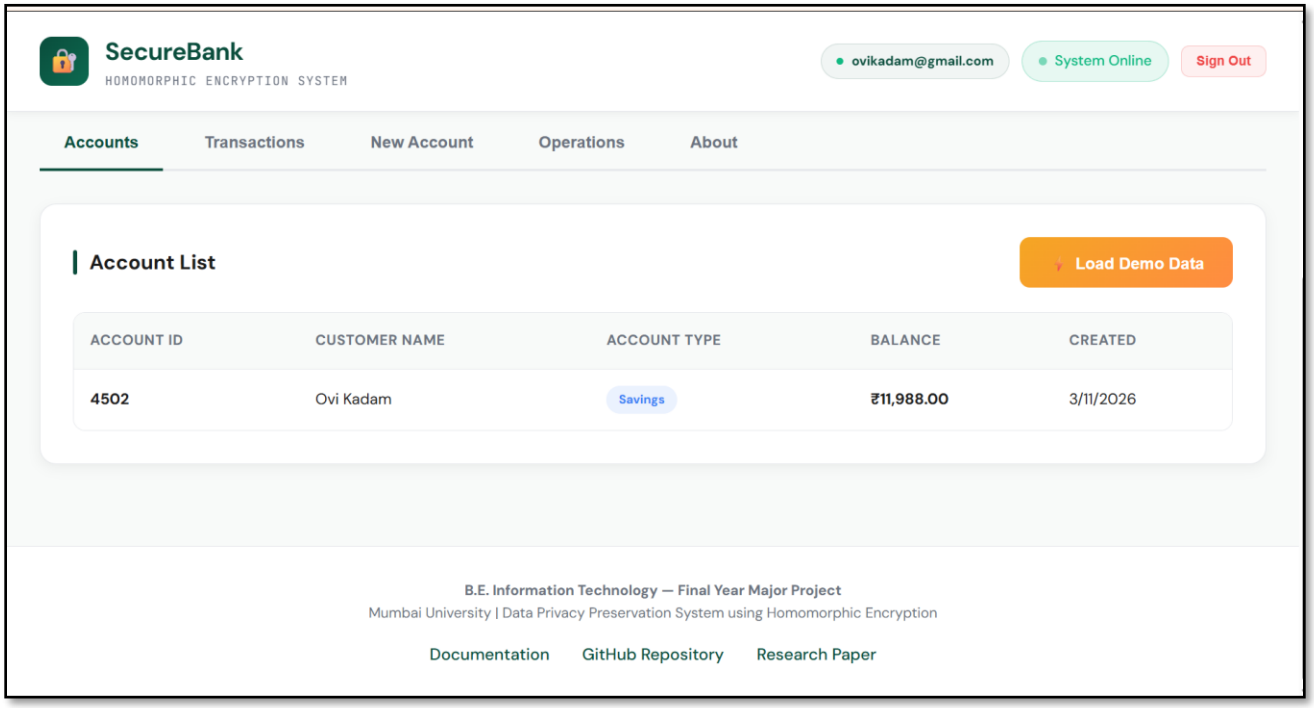

Fig 5.4.3 Accounts Tab

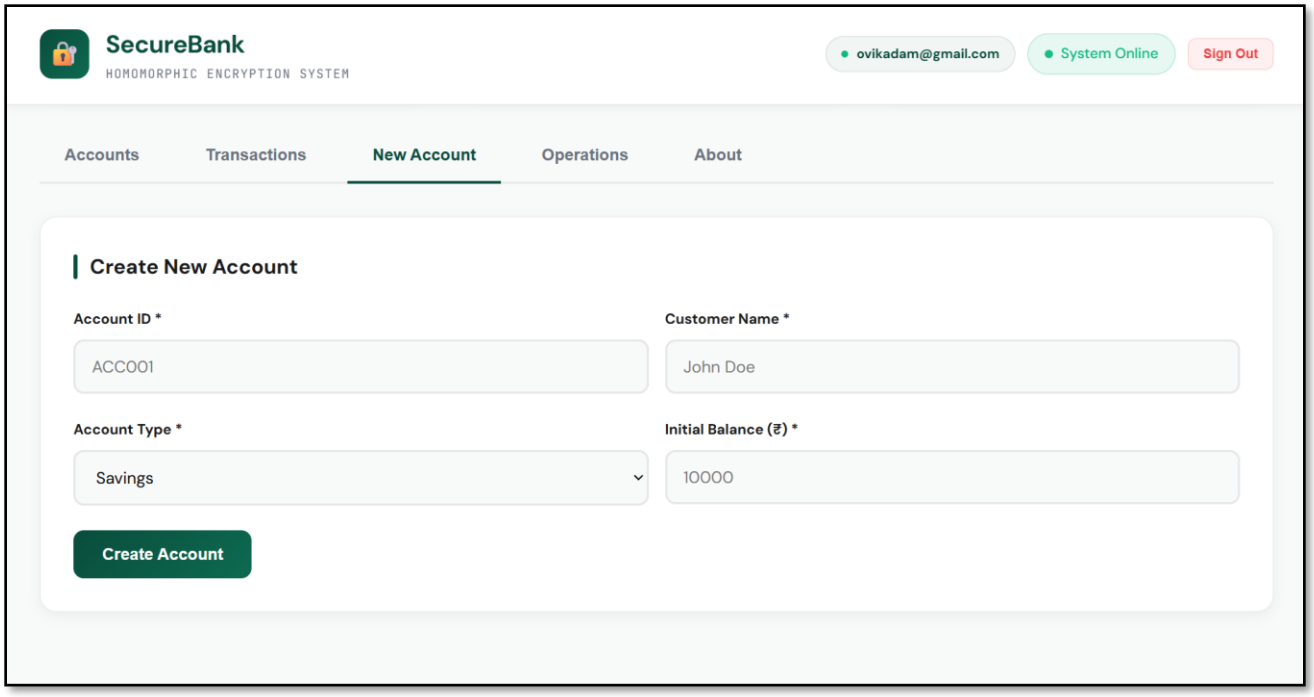

Fig 5.4.4 New Account Creation Form

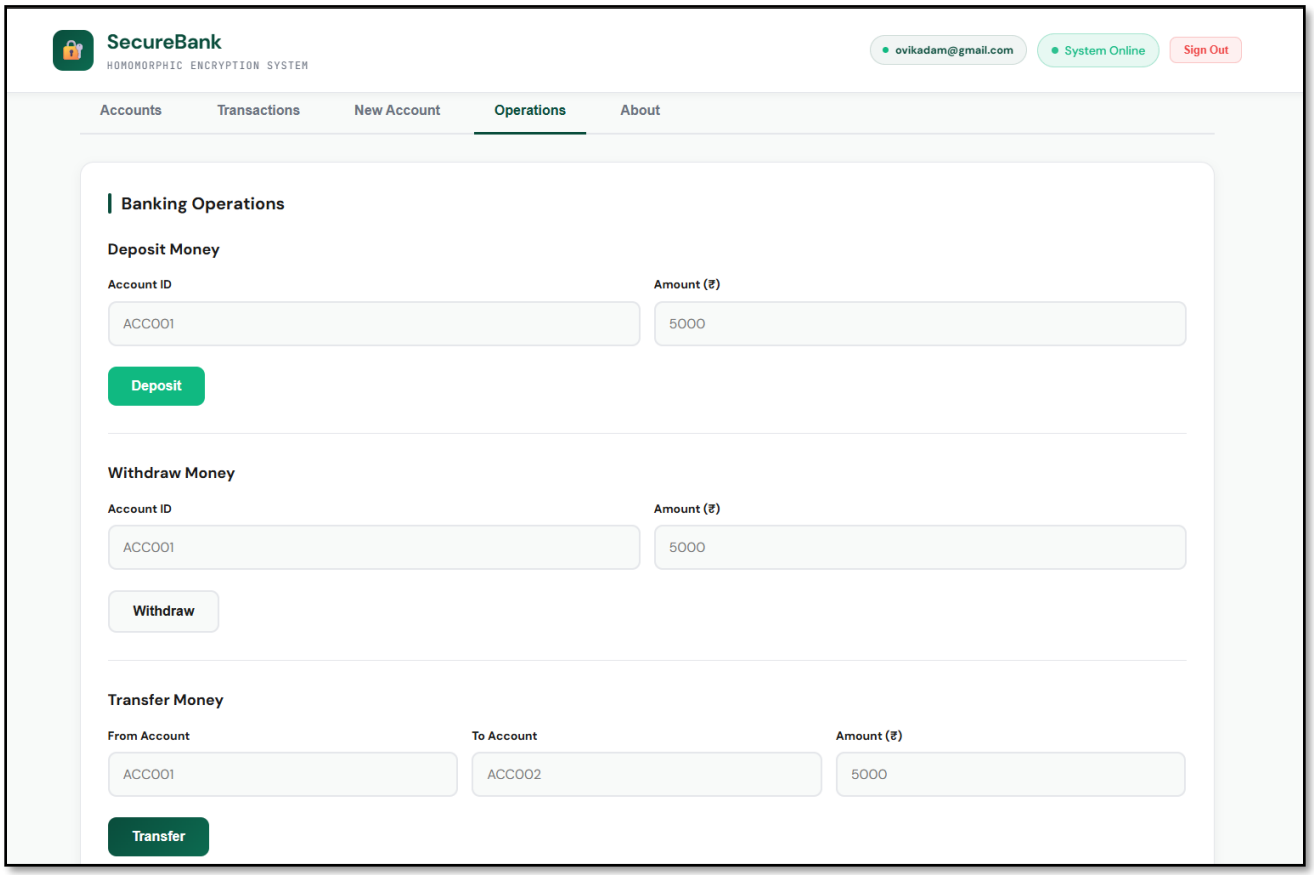

Fig 5.4.5 Deposit, Withdrawal, Transfer Operation

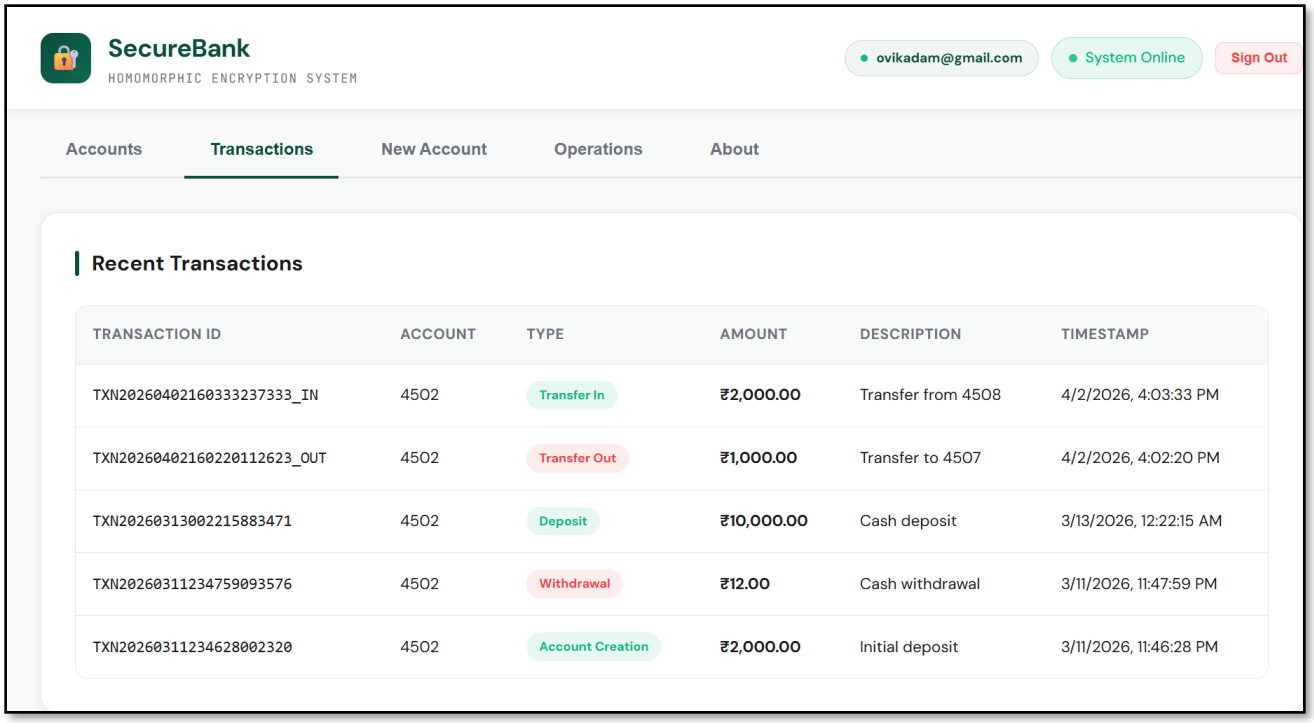

Fig 5.4.6 Transactions Tab

5.5 Performance Benchmarking Suite — paillier_performance_metrics.py

The benchmarking module runs 13 distinct measurements across 50 repetitions each using Python's time.perf_counter() for high-resolution timing. Results include mean, median, standard deviation, min, max, and operations per second. The module also measures correctness across 100 random balance samples and serialises a ciphertext to measure its byte size.

Table 5.5: Performance Benchmark Suite — 13 Metrics Description

Benchmark	Label	What It Measures
1	Key Generation	Time for generate_keypair(512) including two Miller-Rabin prime searches.
2	Encrypt (float)	encrypt_float() for a random balance — includes random r generation and two modular exponentiations.
3	Decrypt (float)	decrypt_float() — one modular exponentiation plus L-function and modular inverse.
4	HE Addition	add_encrypted()single multiplication mod n^2.Near-zero cost.

Benchmark	Label	What It Measures
5	HE Subtraction	Full withdrawal path: encrypt balance + compute negative + add_encrypted.
6	Scalar Multiply×1.05	multiply_encrypted_by_constant() for 5% interest — pow(ct, 105000, n^2).
7	Deposit E2E	encrypt_balance → add_encrypted (deposit) → decrypt_balance. Full round-trip.
8	Withdrawal E2E	encrypt_balance → process_transaction(withdraw) → decrypt_balance.
9	Transfer E2E	Encrypt two balances → debit + credit → decrypt both. Two-account full cycle.
10	Serialization RT	str(ciphertext) + int(str) round-trip — mirrors Firestore storage overhead.
11	Analytics (5 accs)	Total-balance: encrypt 5 balances + 4 homomorphic additions + 1 decrypt.
12	Correctness	100 random balances encrypted and decrypted; absolute error computed. Pass if < ₹0.01.
13	Ciphertext Size	Byte length of a serialized Paillier ciphertext string.

Chapter 6

Advantages and Applications

6.1 Advantages of the Paillier HE Banking System

Table 6.1: Advantages of Paillier HE Banking System

Advantage	Explanation
Zero Plaintext Exposure	Account balances are encrypted before storage. The Flask server computes on ciphertexts for deposits, withdrawals, and transfers — plaintext values are never written to Firestore or logs.
Perfect Accuracy	Paillier's exact integer arithmetic guarantees zero decryption error. Benchmark results show 0.000000 ₹ mean error across 100 test samples (100% pass rate).
Semantic Security	Each encryption of the same balance produces a different ciphertext due to the random blinding factor r. An attacker cannot determine the balance even by encrypting known values.
No Schema Change for Encryption	The Firestore document structure for an account is identical to a non-encrypted system — only the encrypted_balance field changes from float to a ciphertext string. Integration overhead is minimal.
Per-User Cryptographic Isolation	Firebase Auth UID scopes all Firestore queries. Even with the same public key, user A cannot access or modify user B's ciphertexts — enforced at the API and database rule level.

Advantage	Explanation
Cross-User Transfers Without Decryption of Recipient	The account_registry collection allows locating any account globally. Transfers are performed homomorphically; the recipient's balance is never decrypted during a transfer.
Lightweight Dependency	The Paillier implementation uses Python's stdlib only — no third-party crypto library required. This simplifies deployment and reduces the attack surface.
Fast Non-Sensitive Operations	HE Addition (for deposits and analytics) takes ~0.010 ms. Interest calculation (scalar multiplication) takes ~0.184 ms. Only encryption (9.8 ms) and decryption (9.8 ms) involve modular exponentiation.

6.2 Applications

- Privacy-Preserving Loan Eligibility: Banks can compute debt-to-income ratios on encrypted salary and loan data without exposing either figure — using the same homomorphic addition and scalar multiplication primitives.
- Secure Payroll Systems: Employers can compute total payroll, tax deductions, and net salaries across all employees on encrypted salary data, with only the aggregated result decrypted.
- Encrypted Credit Scoring: A credit bureau can compute a score as a weighted sum of encrypted financial indicators (payment history, utilisation, age of accounts) without ever seeing individual values.
- Government Financial Auditing: Auditors can verify that the sum of encrypted departmental budgets equals the declared total — without learning individual departmental allocations.
- Insurance Premium Calculation: Premiums computed as weighted sums of encrypted health indicators without exposing the patient's data to the insurer.
- Privacy-Preserving Tax Filing: Tax authorities can verify aggregate income and deduction totals on encrypted filings without accessing individual taxpayer data.
- Collaborative Financial Analytics: Multiple competing banks can compute joint market statistics (e.g., aggregate loan volumes) on their combined encrypted data without revealing individual institution figures.

Chapter 7

Performance Comparison — Paillier vs. BFV

7.1 Overview

A parallel banking system was implemented using the BFV (Brakerski/Fan-Vercauteren) scheme via TenSEAL to serve as a performance comparison baseline. Both systems execute identical banking operations under identical benchmark conditions (50 runs, same hardware). The following analysis is drawn directly from the benchmark screenshots produced by paillier_performance_metrics.py and bfv_performance_metrics.py.

7.2 Scheme Parameters

Table 7.2: Scheme Parameters — Paillier vs BFV

Parameter	Paillier (Main Project)	BFV (Comparison Only)
Scheme Type	Partially HE (additive)	Leveled FHE (add + multiply)
Mathematical Basis	DCR assumption (factoring hardness)	Ring Learning With Errors (RLWE)
Key Size / Modulus	n = p·q, 1024 bits (p,q each 512 bits)	N=64 polynomial ring; Q≈2^30; T≈2^20
Implementation	Pure Python stdlib (no dependencies)	TenSEAL (wraps Microsoft SEAL C++)
Float Precision Factor	100,000 (5 decimal places)	1023 scaling factor (DELTA)
Ciphertext Size	617 bytes (0.60 KB) as decimal string	1,414 bytes (1.38 KB) serialized

Parameter	Paillier (Main Project)	BFV (Comparison Only)
Storage Format	Decimal integer string in Firestore	Hex string in Firestore
Supported Operations	Unbounded addition, scalar multiply	Leveled: add + multiply (limited depth)

7.3 Timing Benchmark Results

All timings are mean values across 50 runs on the same machine. Lower is better.

Table 7.3: Timing Benchmark Results — Paillier vs BFV (50 Runs)

Operation	Paillier Mean (ms)	BFV Mean (ms)	Winner	Ratio
Key Generation	189.583	0.445	BFV	425× faster
Encrypt (float)	9.821	0.831	BFV	11.8× faster
Decrypt (float)	9.808	0.383	BFV	25.6× faster
HE Addition	0.010	0.009	Tie	~Equal
HE Subtraction	24.477	0.012	BFV	2040× faster
Scalar Multiply	0.184	0.008	BFV	23× faster
Deposit E2E	29.406	2.049	BFV	14.3× faster
Withdrawal E2E	44.021	2.067	BFV	21.3× faster
Transfer E2E	73.536	4.144	BFV	17.7× faster
Serialization RT	0.008	0.025	Paillier	3.1× faster
Analytics (5 accs)	59.289	4.542	BFV	13× faster

7.4 Correctness / Accuracy Comparison

Table 7.4: Correctness / Accuracy Comparison

Metric	Paillier	BFV	Verdict
Samples Tested	100	100	Equal
Mean Absolute Error	₹0.000000	₹2.6163	Paillier wins
Maximum Absolute Error	₹0.000000	₹5.12	Paillier wins
Pass Rate (< ₹0.01 error)	100.0%	0.0%	Paillier wins decisively
Status	PASS	DEGRADED	Paillier wins decisively

The BFV DEGRADED status arises because the tested BFV parameter set (N=64, T≈2^20, DELTA=1023) has insufficient plaintext modulus capacity to represent banking balances scaled to 5 decimal places without overflow and rounding noise. BFV would require a significantly larger T (plaintext modulus) and N (polynomial degree) to achieve banking-grade accuracy — at the cost of much larger ciphertext sizes and key generation time.

7.5 Ciphertext Size Comparison

Table 7.5: Ciphertext Size Comparison

Metric	Paillier	BFV
Serialized Size	617 bytes (0.60 KB)	1,414 bytes (1.38 KB)
Storage Format	Decimal string	Binary (hex-encoded)
Firestore Overhead	~617 chars per field	~2828 hex chars per field

7.6 Comparative Analysis Summary

The comparison shows that Paillier encryption is more suitable for banking applications, primarily due to its perfect accuracy and reliability in handling financial data. It produces zero error in currency computations, ensuring correctness, while BFV shows degraded accuracy with a mean

error of around ₹2.6, which is unacceptable for sensitive financial operations. Although BFV significantly outperforms Paillier in terms of speed (faster encryption/decryption and much quicker key generation), and also supports more advanced operations like addition and multiplication, this comes at the cost of precision and increased implementation complexity. Paillier, on the other hand, offers smaller ciphertext size, faster serialization, and a simpler implementation using standard Python libraries. While both schemes perform similarly in basic homomorphic addition operations, the overall banking correctness verdict clearly favors Paillier, making it the better choice where accuracy and data integrity are critical.

Chapter 8
Conclusion and Future Scope

8.1 Conclusion

This project implements a secure banking system using Paillier Homomorphic Encryption, enabling all operations like deposit, withdrawal, and transfer to be performed directly on encrypted data stored in Firebase without exposing plaintext.

Performance results show that the system is practical, with operations completing within acceptable time limits (29–73 ms), making it suitable for real-world non-real-time banking applications.

The system achieves perfect decryption accuracy with zero error and ensures strong security through semantic encryption, Firebase authentication, and strict data isolation.

Comparison with BFV highlights that although BFV is faster, it lacks accuracy for financial use, making Paillier a better choice. The system is also scalable and can be easily integrated into production environments.

8.2 Future Scope

For the proposed system, Paillier encryption should be enhanced in terms of performance and speed, as it currently lags behind BFV in encryption, decryption, and key generation time despite providing perfect accuracy. Optimizing its computational efficiency will make it more practical for real-time banking applications. On the other hand, BFV requires improvement in accuracy, as it currently produces degraded results in financial computations; ensuring precise and error-free calculations is essential for its reliable use in banking systems.

References

[1] J. Kim and A. Yun, “Secure Fully Homomorphic Authenticated Encryption,” in IEEE Access, vol. 9, pp. 107279–107297, 2021, doi:10.1109/ACCESS.2021.3100852.

[2] H. Cheng, X. Liao, H. Li, Q. Lu and Y. Zhao, “Privacy-Preserving Push- ¨Pull Method for Decentralized Optimization via State Decomposition,”in IEEE Transactions on Signal and Information Processing over Networks, vol. 10, pp. 513–526, 2024, doi: 10.1109/TSIPN.2024.3402430.

[3] R. Wang, J. Lai, Z. Zhang, X. Li, P. Vijayakumar and M. Karuppiah, “Privacy-Preserving Federated Learning for Internet of MedicalThings Under Edge Computing,” in IEEE Journal of Biomedical and Health Informatics, vol. 27, no. 2, pp. 854–865, Feb. 2023, doi:10.1109/JBHI.2022.3157725.

[4] W. Ding, Z. Yan and R. H. Deng, “Privacy-Preserving Data Processing with Flexible Access Control,” in IEEE Transactions on Dependable and Secure Computing, vol. 17, no. 2, pp. 363–376, March–April 2020, doi: 10.1109/TDSC.2017.2786247.

[5] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo and R. H. Deng, “PrivacyPreserving Byzantine-Robust Federated Learning via Blockchain Systems,” in IEEE Transactions on Information Forensics and Security, vol. 17, pp. 2848–2861, 2022, doi: 10.1109/TIFS.2022.3196274.

[6] M. Li, "Leveled Certificateless Fully Homomorphic Encryption Schemes From Learning With Errors," in IEEE Access, vol. 8, pp. 26749-26763, 2020, doi: 10.1109/ACCESS.2020.2971342.

[7] X. Yang, S. Zheng, T. Zhou, Y. Liu and X. Che, "Optimized relinearization algorithm of the multikey homomorphic encryption scheme," in Tsinghua Science and Technology, vol. 27, no. 3, pp. 642-652, June 2022, doi: 10.26599/TST.2021.9010047.

[8] C. Zhou and N. Ansari, "Securing Federated Learning Enabled NWDAF Architecture With Partial Homomorphic Encryption," in IEEE Networking Letters, vol. 5, no. 4, pp. 299-303, Dec. 2023, doi: 10.1109/LNET.2023.3294497.

[9] Y. Yang, S. Zhang, J. Yang, J. Li and Z. Li, "Targeted fully homomorphic encryption based on a Double Decryption Algorithm for polynomials," in Tsinghua Science and Technology, vol. 19, no. 5, pp. 478-485, Oct. 2014, doi: 10.1109/TST.2014.6919824.

[10] Y. Zhu et al., “DAHE: Parameter-Adaptive and Memory Efficient FPGA Acceleration of Homomorphic Encryption,” in IEEE Transactions on Computers, vol. 74, no. 8, pp. 2687–2701, Aug. 2025, doi:10.1109/TC.2025.3569159.

[11] B. Jia, X. Zhang, J. Liu, Y. Zhang, K. Huang and Y. Liang, “Blockchain Enabled Federated Learning Data Protection Aggregation Scheme With Differential Privacy and Homomorphic Encryption in IIoT,” in IEEE Transactions on Industrial Informatics, vol. 18, no. 6, pp. 4049–4058, June 2022, doi: 10.1109/TII.2021.3085960.

[12] N. Kakade and U. Patel, “Secure Secret Sharing Using Homomorphic Encryption,” 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), Kharagpur, India, 2020, pp. 1–7, doi: 10.1109/ICCCNT49239.2020.9225325.

[13] P. Zheng et al., “Accurate and Efficient Privacy-Preserving FeaturenExtraction on Encrypted Images,” in IEEE Transactions on Dependable and Secure Computing, vol. 22, no. 4, pp. 3192 3207, July–Aug. 2025, doi: 10.1109/TDSC.2024.3524121.

[14] M. M. S. Altaee and M. Alanezi, “Enhancing Cloud Computing Security by Paillier Homomorphic Encryption,” International Journal of Electrical and Computer Engineering, vol. 11, no. 2, pp. 1771–1779,2021.

[15] M. Tebaa, K. Zkik and S. El Hajji, “Hybrid Homomorphic Encryption Method for Protecting the Privacy of Banking Data in the Cloud,” International Journal of Security and Its Applications, vol. 9, no. 6, pp. 61–70, 2015.

[16] A. Laouid, M. Kara, M. Hammoudeh, H. Zelaci and A. K. Bashir, "An Edge-Based Quantum-Secure Fully Homomorphic Encryption Scheme," in IEEE Transactions on Consumer Electronics, vol. 71, no. 3, pp. 8396-8406, Aug. 2025, doi: 10.1109/TCE.2025.3593988.

[17] G. Tu, W. Liu, T. Zhou, X. Yang and F. Zhang, "Concise and Efficient Multi-Identity Fully Homomorphic Encryption Scheme," in IEEE Access, vol. 12, pp. 49640-49652, 2024, doi: 10.1109/ACCESS.2024.3384247.

[18] G. Wu et al., "E-Booster: A Field-Programmable Gate Array-Based Accelerator for Secure Tree Boosting Using Additively Homomorphic Encryption," in IEEE Micro, vol. 43, no. 5, pp. 88-96, Sept.-Oct. 2023, doi: 10.1109/MM.2023.3293845.

[19] T. Shen, F. Wang, K. Chen, K. Wang and B. Li, "Efficient Leveled (Multi) Identity-Based Fully Homomorphic Encryption Schemes," in IEEE Access, vol. 7, pp. 79299-79310, 2019, doi: 10.1109/ACCESS.2019.2922685.

[20] K. Sinha, P. Majumder and S. K. Ghosh, "Fully Homomorphic Encryption based Privacy-Preserving Data Acquisition and Computation for Contact Tracing," 2020 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), New Delhi, India, 2020, pp. 1-6, doi: 10.1109/ANTS50601.2020.9342834

Achievements / Publications by the Candidates

Title: "Bank Data Privacy Preservation System using Homomorphic Encryption"

Authors: Arpita Bhosle, Aarya Bhujbal, Aditya Desai, Prathamesh Wankhede

Target Conference: 2026 6th International Conference on Intelligent Technologies (CONIT)

Date: 19–21 June 2026

Venue: Belgavi, India (Hybrid — In-person and Virtual)

Organizer: IEEE — Sponsored by Bangalore Section, Bangalore Section IT Chapter, KLE Institute of Technology, Hubli

Status: **Manuscript prepared; submission in progress**

Acknowledgement

We are thankful to “Bharati Vidyapeeth College of Engineering”, for considering our Project Report and guide us through the various stages of the project report. It gives us immense pleasure to express our sincere gratitude towards our Project guide HOD, Prof. S.N. Mhatre and Co-ordinator Prof. Archana Chavan for her guidance in selecting the final year project and also for providing us with all the details.

We are deeply indebted to our respected Principal, Dr. Sandhya Jadhav and HOD, Prof. S.N. Mhatre for giving us this valuable opportunity to do this project and we express our hearty thanks to them for their assistance without which it would have been difficult in finishing this final year project report successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff of Information Technology Department for their constant encouragement, support and timely help throughout the final year project report work.